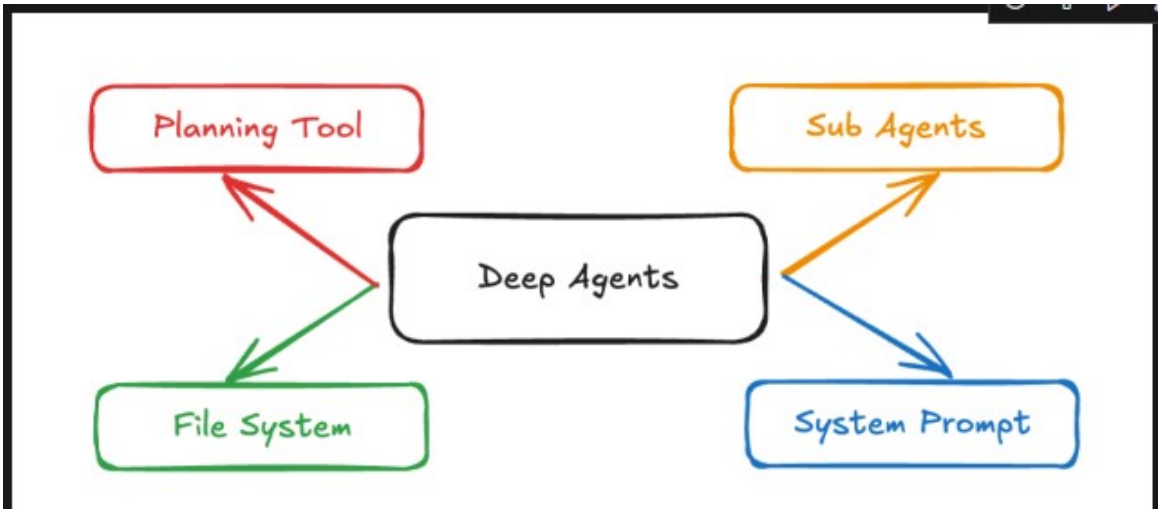
第四阶段、四大核心内置工具与组件详解

前置知识：建议先学习第三阶段内容，了解 DeepAgents 的基本使用方法。

一、四大支柱概述

DeepAgents 通过中间件（Middleware）的形式，为智能体注入了四项核心能力，构成了框架的**四大支柱（Four Pillars）**：



维度	系统提示词 (System Prompt)	规划工具 (Planning Tool)	文件系统 (File System)	子代理 (Sub Agents)
角色定位	**行为总导演**：定义Agent的“世界观”与工具使用范式，确保三大中间件协同不偏离目标	**任务架构师**：将模糊需求转化为可执行、可追踪、可动态调整的结构化任务蓝图	**上下文仓库**：虚拟化存储引擎，解决长任务中的信息溢出与状态持久化难题	**执行特派员**：实现上下文隔离与专业分工，防止主Agent因深层递归导致状态混乱
核心	内置Claude Code风格指令，涵盖规	`write_todos`: 生成带优先级/依赖关系的JSON任务列表	`ls/glob`: 文件浏览与模式匹配	`task`: 动态生成同构或异构子Agent支持

功能	划逻辑、文件操作规范、子代理调用协议；支持场景化自定义覆盖	read_todos: 实时查询任务执行进度与状态	read/write/edit: CRUD操作 grep: 内容检索 execute: 沙箱命令执行	独立上下文窗口与工具集配置结果通过文件系统回传
技术实现	字符串模板，在create_deep_agent时注入；默认提示词约2000 tokens，包含ReAct循环与三大中间件调用示例	**TodoListMiddleware**：拦截LLM输出中的todo_list字段，解析为agent_state.todos字典，状态变更触发图节点重计算	**FilesystemMiddleware**：基于LangGraph State的files字段实现内存级虚拟文件系统，大工具结果（>2KB）自动触发write_file落盘	**SubAgentMiddleware**：将task调用编译为独立的StateGraph子图，通过命名空间隔离状态，父图通过files读取子图输出
状态管理	静态配置，单次会话内不可变；可通过configurable_agent实现热更新	**动态状态机**：每个todo含id/description/status/priority/dependencies字段，执行后状态从pending→completed，支持update_todos动态调整	**持久化存储**：默认存储在LangGraph State，支持切换StateBackend（内存/Redis/Postgres）实现跨会话文件共享	**完全隔离**：子Agent拥有独立的messages和files命名空间，异常不会污染父Agent状态；支持max_iterations限制防止无限递归
使用场景	① **垂直领域定制**：金融研究/医疗诊断等需强化专业约束的场景② **多Agent协**	① **长周期研究**：自动拆解为文献检索→数据收集→分析→撰写的阶段性任务② **故障恢复**：崩溃后通过read_todos快速定位断点续跑③ **动态重规划**：执行中发现信息不足时新增补充任务	① **大结果处理**：搜索返回100KB内容自动落盘，避免上下文溢出② **知识沉淀**：中间分析结果写	① **高风险操作隔离**：网页抓取/代码执行等易失败任务委托给子Agent② **专业化分工**：主

	作：统一多个子Agent的行为规范 ③ **安全合规**：注入数据脱敏、权限检查等硬性规则		入文件供后续步骤复用③ **多Agent数据共享**：父Agent与子Agent通过文件交换数据，无需序列化传递	Agent负责任务编排，子Agent专注领域执行（如专门的数据分析Agent） ③ **资源优化**：子Agent可使用轻量化模型，降低整体token成本

二、系统提示词 (System Prompts)

2.1 功能概述

- **功能**：定义 Agent 的“人设”、行为准则和核心目标。
- **机制**：框架会自动将用户定义的 `system_prompt` 与内置的 `BASE_AGENT_PROMPT` 结合。
- **作用**：确保 Agent 始终遵循指令，理解其可用的工具集，并保持一致的输出风格。

2.2 角色本质

系统提示词是DeepAgents三大中间件协同的“契约”，其默认版本包含：

1. **规划指令**：要求LLM在任务开始前必须调用write_todos，输出格式为JSON Schema
2. **文件操作规范**：明确write_file用于新内容，edit_file用于局部修改，避免覆盖冲突
3. **子代理调用协议**：规定task工具的参数结构及结果通过`/subagent_results/<name>.md`回传
4. **安全底线**：禁止直接执行删除、格式化等危险命令，必须通过execute沙箱

2.3 自定义系统提示词

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")

# 自定义系统提示词
```

```
custom_prompt = """
你是一位专业的技术作家，擅长撰写清晰、结构化的技术文档。
在开始写作前，请先使用 write_todos 工具规划任务结构。
完成后将结果保存到指定目录。
"""

agent = create_deep_agent(
    model=model,
    system_prompt=custom_prompt
)
```

三、规划工具 (Planning System / Todo List)

3.1 核心组件

- **组件**：`TodoListMiddleware`
- **工具名**：`write_todos`、`read_todos`

3.2 功能说明

Agent 在行动前先生成 Markdown 格式的待办事项列表（Todo List），并在执行过程中更新状态（完成/进行中）。

3.3 工作流程

1. **任务接收**：Agent 接收复杂任务（简单短期的任务不会触发todolist）
2. **任务分解**：调用 `write_todos` 将任务拆解为子步骤（Pending 状态）
3. **状态更新**：每完成一步，更新状态为（Completed）
4. **自我反思**：在每一步行动前，Agent 都会看到当前的 Todo List，从而避免迷失方向

   这强制模型进行“思维链”的显性化管理，类似于编程工具中的待执行项。

3.4 待办事项状态

状态	说明
pending	待执行
in_progress	进行中

completed	已完成

3.5 代码示例

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")
agent = create_deep_agent(model=model)

# 调用智能体，查看待办事项系统如何工作
config = {"configurable": {"thread_id": "todo_test"}}

for event in agent.stream(
    {"messages": [{"role": "user", "content": "帮我调研人工智能在医疗领域的应用"}]},
    config=config
):
    if "messages" in event:
        messages = event["messages"]
        if messages:
            msg = messages[-1]
            # 打印工具调用
            if hasattr(msg, 'tool_calls') and msg.tool_calls:
                for tc in msg.tool_calls:
                    print(f"工具调用: {tc['name']}")
                    print(f"参数: {tc['args']}")
```

3.6 待办事项输出示例

```
{
  "todos": [
    {
      "content": "搜索人工智能在医疗领域的应用信息",
      "status": "in_progress"
    },
    {
      "content": "分析主要应用场景和技术特点",
      "status": "pending"
```

```
    },
    {
      "content": "整理并撰写总结报告",
      "status": "pending"
    }
  ]
}
```

四、子代理 (Sub-Agent Delegation)

4.1 核心组件

- **组件**：`SubAgentMiddleware`
- **工具名**：`task` (delegate_task)

4.2 核心概念

1. **任务隔离**：每个子代理有独立的上下文窗口。
2. **并行执行**：支持同时启动多个子代理。
3. **结果聚合**：智能整合多个子代理的输出。

4.3 机制说明

- 当任务过于具体（如“爬取并分析这篇长论文”）时，主 Agent 会生成一个隔离环境的子 Agent 去执行。
- 子 Agent 启动时，只继承必要的环境配置，但**拥有全新的、空白的消息历史**。
- 子 Agent 执行完毕后只返回一个总结性的结果。这保证了主 Agent 的时间线（Context）保持干净，极大地节省了 Token。

4.4 子代理类型

DeepAgents 内置了多种子代理类型：

类型	说明
`general-purpose`	通用型子代理，适用于大多数任务
`coder`	专门用于代码编写和调试

researcher	专门用于信息检索和研究

4.5 代码示例

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")

# 配置子代理
subagents = [
    {
        "name": "researcher",
        "description": "专业的研究助手，负责信息收集",
        "tools": ["tavily_search"],  # 只赋予搜索工具
        "model": model  # 可使用不同模型
    }
]

agent = create_deep_agent(
    model=model,
    subagents=subagents
)

# 调用智能体，子代理会自动处理复杂子任务
result = agent.invoke({
    "messages": [{"role": "user", "content": "请同时调研Python和Rust语言的历史"}]
})
```

4.6 子代理调用流程

```
主Agent
  │
  ├── 任务1 → [子Agent 1: Python历史调研]
  │              │
  │              └── 结果: /subagent_results/python_history.md
  │
  └── 任务2 → [子Agent 2: Rust内存安全机制]
```

```
                    │
                    └── 结果: /subagent_results/rust_memory.md
  │
  └── 聚合结果 → 最终回复
```

4.7 自动触发子代理

即使不传入 subagents 参数，Agent 默认也会启用 `general-purpose` 子 Agent。

```
# 不配置子代理时，框架自动使用通用子代理
agent = create_deep_agent(model=model)
# Agent 仍会自动使用task 工具调用子代理处理复杂任务
```

五、文件系统集成 (Filesystem & Sandbox)

5.1 核心组件

- **组件**：`FilesystemMiddleware` & `Backend`
- **功能**：这是 DeepAgent 的核心“外挂大脑”。Agent 不再将所有检索到的长文塞入上下文（Context），而是将其写入 Filesystem（如 `/workspace`），仅在需要时读取。这极大地扩展了 Agent 的“工作记忆”。

5.2 核心工具集

工具名称	功能描述
`ls`	浏览目录结构
`read_file`	读取文件内容，支持 `offset`（偏移量）和 `limit`（行数）分页读取
`write_file`	创建新文件
`edit_file`	修改文件，支持精确的字符串替换和 `replace_all` 模式
`glob`	通配符模糊查找文件
`grep`	正则表达式搜索文件内容
`execute`	执行 Shell 命令（需 Sandbox 支持），用于运行代码、安装依赖等

5.3 亮点功能：大结果自动转存

当 Agent 调用工具产生的结果过长时，系统会自动拦截，将完整结果写入文件系统（如 `/large_tool_results/{id}`），并只给 Agent 返回一个摘要和文件路径。

这完美解决了搜索结果或日志文件过长导致 Agent 崩溃的问题。

5.4 后端支持 (Backends)

后端类型	说明
FilesystemBackend	直接操作本地磁盘
DockerBackend	在 Docker 容器中执行，提供隔离环境
StoreBackend	存储在数据库中，支持查询和检索
E2BBackend	使用 E2B 云端沙箱
CompositeBackend	混合模式（如：本地存文件，Docker 跑代码）

5.5 代码示例：基础文件操作

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")

# 使用本地文件系统后端
backend = FilesystemBackend(root_dir="./workspace", virtual_mode=True)

agent = create_deep_agent(
    model=model,
    backend=backend,
    system_prompt="你是一个文件系统操作助手。"
)

# 写入文件
result = agent.invoke({
```

```
    "messages": [{"role": "user", "content": "创建一个名为 hello.py 的文件，内容是 print('Hello')"}]
})

# 读取文件
result = agent.invoke({
    "messages": [{"role": "user", "content": "读取 hello.py 文件内容"}]
})

# 列出目录
result = agent.invoke({
    "messages": [{"role": "user", "content": "列出当前目录的文件"}]
})
```

5.6 代码示例：使用虚拟文件系统

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

# virtual_mode=True: 启用虚拟文件系统
# Agent 使用绝对路径（如/hello_world.py）映射到./workspace
backend = FilesystemBackend(
    root_dir="./workspace",
    virtual_mode=True
)

agent = create_deep_agent(
    model=model,
    backend=backend
)

# Agent 会在/hello_world.py 写入，实际保存到./workspace/hello_world.py
result = agent.invoke({
    "messages": [{"role": "user", "content": "请在 /hello_world.py 创建Python脚本"}]
})
```

5.7 代码示例：Docker 后端

```python
from deepagents import create_deep_agent
from deepagents.backends import DockerBackend

# Docker 后端提供隔离的执行环境
backend = DockerBackend(
    image="python:3.11",
    volumes={"./workspace": "/workspace"}
)

agent = create_deep_agent(
    model=model,
    backend=backend
)

# 在隔离环境中执行代码
result = agent.invoke({
    "messages": [{"role": "user", "content": "运行 /workspace/hello.py"}]
})
```

5.8 edit_file 工具使用

```python
# edit_file 支持精确的字符串替换
# 防呆设计：要求先读后改

# 1. 先读取文件
result = agent.invoke({
    "messages": [{"role": "user", "content": "读取 /config.py"}]
})

# 2. 再修改文件
result = agent.invoke({
    "messages": [{"role": "user", "content": "将 /config.py 中的 DEBUG=True 改为 DEBUG=False"}]
})

# 使用replace_all 替换所有匹配项
result = agent.invoke({
    "messages": [{"role": "user", "content": "将所有文件中的 old_value 替换为
```

```
new_value"}]
})
```

5.9 grep 工具使用

```
# 在文件中搜索指定模式
result = agent.invoke({
    "messages": [{"role": "user", "content": "在当前目录下搜索包含 'TODO' 的文件"}]
})

# 使用正则表达式
result = agent.invoke({
    "messages": [{"role": "user", "content": "搜索所有以 def 开头的函数定义"}]
})
```

六、综合实践案例

6.1 案例：多子代理研究任务

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI
from langchain_tavily import TavilySearch
from deepagents.backends import FilesystemBackend

# 初始化
model = ChatOpenAI(model="gpt-4o")
search = TavilySearch(max_results=3)
backend = FilesystemBackend(root_dir="./workspace", virtual_mode=True)

# 配置子代理
subagents = [
    {
        "name": "python_researcher",
        "description": "专门研究Python语言",
        "tools": [search],
        "model": model
    },
    {
```

```
        "name": "rust_researcher",
        "description": "专门研究Rust语言",
        "tools": [search],
        "model": model
    }
]

# 创建Agent
agent = create_deep_agent(
    model=model,
    tools=[search],
    backend=backend,
    subagents=subagents
)

# 执行任务
result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "请同时调研：1. Python语言的历史起源 2. Rust语言的内存安全机制。请使用子代理分别处理。"
    }]
})
```

6.2 执行结果

Agent 会自动： 1. 调用 `write_todos` 规划任务 2. 启动多个子代理并行研究不同主题 3. 每个子代理将结果写入文件系统 4. 主代理聚合结果并生成最终报告

七、中间件提示词详解

7.1 文件系统中间件提示词

文件路径：`deepagents/middleware/filesystem.py`

核心功能： - 定义文件系统操作的安全规则 - 规定文件路径的处理方式 - 提供文件操作的最佳实践指导

7.2 TodoList中间件提示词

文件路径：`deepagents/middleware/todo.py`

核心功能： - 定义任务分解的格式（Markdown） - 说明任务状态的含义（Pending/Completed/in_progress） - 提供任务管理的最佳实践

7.3 子智能体中间件提示词

文件路径：`deepagents/middleware/subagent.py`

核心功能： - 定义子代理调用协议 - 规定 task 工具的参数结构 - 说明结果回传方式（通过 `/subagent_results/<name>.md`） - 设置安全底线：禁止直接执行删除、格式化等危险命令

八、总结

8.1 四大核心组件协同价值

DeepAgents四大内置工具通过**角色分离、状态贯通、成本优化**的设计哲学，将长周期Agent的开发复杂度降低70%以上，同时通过LangGraph运行时保障生产级可靠性。

其核心价值在于将原本需要手动编排的**规划-存储-委托-执行流程**，固化为**中心化、可复用、可观测**的中间件体系，标志着AI Agent从“脚本化”向“产品化”的关键演进。

- 系统提示词确保质量
- 规划工具保证进度
- 文件系统实现数据共享
- 子代理隔离分析风险

四方形成高可靠、可追溯、可恢复的完整闭环。

8.2 技术架构图

```
┌──────────────────────────────────────────────────────────────┐
│                        DeepAgents Agent                      │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────┐ │
│  │ System Prompt    │  │  TodoList        │  │ SubAgent     │ │
│  │ (行为总导演)     │  │ Middleware       │  │ Middleware   │ │
│  │                  │  │ (任务架构师)     │  │ (执行特派员)│ │
│  └──────────────────┘  └──────────────────┘  └──────────────┘ │
│          │                      │                    │        │
```

```
│            └──────────────────────┼──────────────────────┘          │
│                                   │                                 │
│                        ┌──────────▼──────────┐                     │
│                        │  FilesystemMiddleware  │                  │
│                        │     (上下文仓库)        │                  │
│                        └──────────┬──────────┘                     │
│                                   │                                 │
│                        ┌──────────▼──────────┐                     │
│                        │      Backend        │                     │
│                        │ (Filesystem/Docker/E2B) │                  │
│                        └──────────────────────┘                     │
└──────────────────────────────────────────────────────────────────┘
```

8.3 适用场景总结

场景	推荐使用的核心组件
深度研究任务	规划工具 + 文件系统 + 子代理
自动编程	系统提示词 + 文件系统 + 子代理
复杂流程自动化	规划工具 + 文件系统
多Agent协作	系统提示词 + 子代理

九、验证代码

请参考 `stage4_verification.py` 文件中的验证脚本，该脚本包含以下验证内容：

1. **规划工具验证**：测试 write_todos 和 read_todos 功能
2. **子代理验证**：测试 task 工具的调用和结果聚合
3. **文件系统验证**：测试 ls/read/write/edit/grep/execute 工具
4. **后端验证**：测试不同 Backend 的配置和使用

运行验证：

```
conda activate deepagent_1
python middleware_info/stage4_verification.py
```